

Universidad de Buenos Aires

Álgebra Lineal Computacional

APUNTE

**Repaso: representación binaria, enteros
y el camino hacia punto flotante**

18 de agosto de 2026

Índice

1. ¿Qué es un bit?	2
2. El sistema binario (base 2)	2
3. Conversión de binario a decimal	2
4. Conversión de decimal a binario	3
4.1. Método de las divisiones sucesivas	3
4.2. Método de las potencias de 2 (resta sucesiva)	3
5. Bits, nibbles y bytes	4
6. Representación de enteros sin signo	4
6.1. Ejemplos de rangos según la cantidad de bits	5
7. El problema del signo: cómo representar enteros negativos	5
8. De los enteros a los números con parte fraccionaria	5
8.1. La idea: notación científica, pero en base 2	5
8.2. Por qué existe el estándar IEEE 754	6
8.3. Normalización	6
8.4. Estructura de un número IEEE 754	7
9. Cómo convertir un número decimal a IEEE 754	7

1. ¿Qué es un bit?

Una computadora, a nivel físico, trabaja con circuitos que pueden estar en dos estados: hay corriente o no hay corriente, un voltaje alto o un voltaje bajo. Para representar esto matemáticamente usamos dos símbolos: 0 y 1.

Un **bit** (de *binary digit*, dígito binario) es la unidad mínima de información y puede valer 0 o 1. Con un solo bit podemos distinguir entre dos posibilidades, por ejemplo apagado o prendido, falso o verdadero, no o sí.

Si queremos representar más de dos posibilidades necesitamos combinar varios bits. Con n bits, la cantidad de combinaciones distintas que podemos formar es

$$2^n$$

porque cada bit tiene 2 valores posibles y hay n bits, cada uno independiente del resto. Por ejemplo, con 2 bits tenemos $2^2 = 4$ combinaciones posibles: 00, 01, 10, 11. Con 3 bits tenemos $2^3 = 8$ combinaciones: 000, 001, 010, 011, 100, 101, 110, 111.

Esta cuenta, 2^n , va a aparecer todo el tiempo cuando hablemos de rangos de representación.

2. El sistema binario (base 2)

Estamos acostumbrados a contar en **base 10**: usamos diez símbolos, del 0 al 9, y cada posición de un número representa una potencia de 10. Por ejemplo el número 4327 en realidad significa

$$4327 = 4 \cdot 10^3 + 3 \cdot 10^2 + 2 \cdot 10^1 + 7 \cdot 10^0$$

Cada dígito multiplica a una potencia de 10 según el lugar donde está escrito, contando desde la derecha y empezando en el exponente 0.

El sistema **binario** funciona exactamente igual, pero con base 2 en lugar de base 10. Solo tenemos dos símbolos disponibles, 0 y 1, y cada posición representa una potencia de 2. Por ejemplo el número binario 101101 significa

$$1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

Para no confundirnos entre bases, cuando haga falta vamos a escribir el subíndice que indica la base, por ejemplo 101101_2 para el binario y 45_{10} para el decimal equivalente.

3. Conversión de binario a decimal

Convertir de binario a decimal es directamente aplicar la definición de la sección anterior: multiplicar cada bit por la potencia de 2 que le corresponde según su posición, y sumar todo.

Ejemplo. Convertir 101101_2 a decimal.

Numeramos las posiciones de derecha a izquierda empezando en 0:

bit:	1	0	1	1	0	1
posición:	5	4	3	2	1	0

Entonces

$$\begin{aligned} 101101_2 &= 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 \\ &= 32 + 0 + 8 + 4 + 0 + 1 = 45_{10} \end{aligned}$$

Es decir, solo sumamos las potencias de 2 correspondientes a los bits que valen 1, e ignoramos los que valen 0.

Estas son las primeras potencias de 2:

$$2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 8, 2^4 = 16, 2^5 = 32$$
$$2^6 = 64, 2^7 = 128, 2^8 = 256, 2^9 = 512, 2^{10} = 1024$$

4. Conversión de decimal a binario

Ahora vamos en el sentido inverso: dado un número decimal, ¿cómo lo escribimos en binario? Hay dos métodos equivalentes, veamos los dos porque cada uno resulta más cómodo según el caso.

4.1. Método de las divisiones sucesivas

Se divide el número por 2 repetidamente, anotando el resto en cada paso, hasta llegar a un cociente 0. El número binario se arma leyendo los restos de abajo hacia arriba (del último al primero).

Ejemplo. Convertir 37_{10} a binario.

$$\begin{array}{ll} 37 \div 2 = 18 & \text{resto } 1 \\ 18 \div 2 = 9 & \text{resto } 0 \\ 9 \div 2 = 4 & \text{resto } 1 \\ 4 \div 2 = 2 & \text{resto } 0 \\ 2 \div 2 = 1 & \text{resto } 0 \\ 1 \div 2 = 0 & \text{resto } 1 \end{array}$$

Leyendo los restos de abajo hacia arriba obtenemos 100101. Entonces

$$37_{10} = 100101_2$$

Podemos verificar el resultado convirtiendo de nuevo a decimal como en la sección anterior:

$$1 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 32 + 4 + 1 = 37 \checkmark$$

4.2. Método de las potencias de 2 (resta sucesiva)

Este método consiste en buscar la mayor potencia de 2 que sea menor o igual al número, restarla, y repetir con lo que queda, hasta llegar a 0.

Ejemplo. Convertir 37_{10} a binario.

La mayor potencia de 2 menor o igual a 37 es $32 = 2^5$. Restamos: $37 - 32 = 5$.

La mayor potencia de 2 menor o igual a 5 es $4 = 2^2$. Restamos: $5 - 4 = 1$.

La mayor potencia de 2 menor o igual a 1 es $1 = 2^0$. Restamos: $1 - 1 = 0$, terminamos.

Encontramos que $37 = 2^5 + 2^2 + 2^0$. Ahora escribimos un 1 en las posiciones 5, 2 y 0, y un 0 en el resto de las posiciones entre 0 y 5:

$$\begin{array}{rcccccc} \text{posición:} & 5 & 4 & 3 & 2 & 1 & 0 \\ \text{bit:} & 1 & 0 & 0 & 1 & 0 & 1 \end{array}$$

Obtenemos de nuevo 100101_2 , el mismo resultado que con el otro método. Usá el que te resulte más cómodo, ambos son válidos.

5. Bits, nibbles y bytes

Ya vimos que un bit es un 0 o un 1. A partir de ahí se definen algunas agrupaciones de bits que se usan constantemente:

- **Nibble:** 4 bits juntos. Permite representar $2^4 = 16$ valores distintos.
- **Byte:** 8 bits juntos (equivale a 2 nibbles). Permite representar $2^8 = 256$ valores distintos.

El byte es la unidad básica con la que casi siempre se trabaja en una computadora. Cuando decimos que un dato “ocupa 8 bits” o “ocupa un byte” estamos diciendo que ese dato se guarda usando 8 dígitos binarios, y por lo tanto puede tomar cualquiera de $2^8 = 256$ valores distintos, ni uno más.

Así se ve un byte que guarda el número 45 que convertimos antes (101101_2 , completado con ceros a la izquierda hasta ocupar los 8 bits):

7	6	5	4	3	2	1	0
0	0	1	0	1	1	0	1

Cada casillero es un bit, y el número que está arriba indica la posición (la potencia de 2) a la que corresponde, igual que en las tablas de las secciones anteriores.

Esta idea es la que vamos a usar constantemente en lo que sigue: la cantidad de bits que usamos para representar algo determina cuántos valores distintos podemos distinguir, y por lo tanto qué rango de números podemos representar.

6. Representación de enteros sin signo

Supongamos que queremos representar números enteros no negativos ($0, 1, 2, 3, \dots$) usando n bits, sin reservar ningún bit para el signo. A esto se lo llama representación **sin signo** (*unsigned*).

Con n bits tenemos 2^n combinaciones posibles. La forma más natural de usarlas es asignar a cada combinación binaria el número que le corresponde según la conversión binario-decimal de la sección 3, empezando desde 0 (todos los bits en 0) hasta el mayor valor posible (todos los bits en 1).

El valor más chico representable es

$$00 \dots 0_2 = 0_{10}$$

El valor más grande representable con n bits todos en 1 es

$$11 \dots 1_2 = 2^{n-1} + 2^{n-2} + \dots + 2^1 + 2^0 = 2^n - 1$$

(esta última igualdad es la suma de una serie geométrica).

Entonces, con n bits sin signo, el rango de enteros representables es

$$[0, 2^n - 1]$$

y hay exactamente 2^n valores en ese rango, ni uno más ni uno menos, porque cada combinación de bits corresponde a un único número.

6.1. Ejemplos de rangos según la cantidad de bits

Cantidad de bits (n)	Cantidad de valores (2^n)	Rango sin signo
6	64	[0, 63]
8	256	[0, 255]
16	65 536	[0, 65 535]
32	4 294 967 296	[0, 4 294 967 295]
64	$\approx 1,8 \times 10^{19}$	[0, $2^{64} - 1$]

La pregunta “¿cuándo puedo representar un número con 6 bits, con 32, con 64?” se contesta directamente con esta tabla: un número x no negativo se puede representar con n bits sin signo si y solo si $0 \leq x \leq 2^n - 1$. Por ejemplo el número 200 no entra en 6 bits (el máximo es 63), pero sí entra en 8 bits (el máximo es 255), porque $200 \leq 255$.

7. El problema del signo: cómo representar enteros negativos

Hasta acá solo representamos números no negativos. Pero necesitamos también representar números negativos, y ahí aparece un problema nuevo: hay que usar parte de la información (parte de los bits) para indicar el signo, además de la magnitud del número.

Una forma directa de resolver esto es reservar el bit más a la izquierda (el bit más significativo) para indicar el signo (0 para positivo, 1 para negativo), y usar los $n-1$ bits restantes para representar la magnitud (el valor absoluto) en binario normal. A este esquema se lo llama **signo y magnitud**.

Por ejemplo con 8 bits, el número -5 se escribiría como el signo 1 seguido de la magnitud de 5 en 7 bits (0000101):

1 0000101

El problema de este esquema es que el cero tiene dos representaciones distintas: 00000000 (cero positivo) y 10000000 (cero negativo), que matemáticamente deberían ser el mismo número. Esto complica las operaciones aritméticas y desperdicia una combinación de bits. Por eso en la práctica las computadoras usan otros esquemas de codificación (que no vamos a desarrollar acá), pero la idea central se mantiene: siempre hace falta reservar parte de la información para indicar el signo.

8. De los enteros a los números con parte fraccionaria

Todo lo que vimos hasta ahora sirve para representar números enteros, sin parte decimal. Pero en la práctica necesitamos representar también números como 3,14, 0,001 o 1 500 000,75, que tienen parte fraccionaria, y además necesitamos poder representar números enormes (como 10^{300}) o diminutos (como 10^{-300}) sin usar una cantidad de bits absurdamente grande.

Con enteros esto no se puede: si quisiéramos representar 10^{300} como entero exacto necesitaríamos muchísimos más de 64 bits. Necesitamos otra estrategia de representación, que sacrifique algo de precisión a cambio de poder cubrir un rango enorme de valores con una cantidad fija de bits. A esa estrategia se la llama **punto flotante**.

8.1. La idea: notación científica, pero en base 2

En base 10 estamos acostumbrados a la notación científica: en lugar de escribir 300 000 escribimos

$$3,0 \times 10^5$$

Acá separamos el número en dos partes: una **mantisa** (3,0) que tiene un único dígito distinto de cero antes de la coma, y un **exponente** (5) que nos dice cuánto hay que correr la coma decimal para recuperar el número original.

La idea de punto flotante es hacer exactamente lo mismo pero en base 2. Cualquier número (salvo el 0) se puede escribir de forma única como

$$\pm 1.f \times 2^e$$

donde:

- El signo ($\pm$) indica si el número es positivo o negativo, y se guarda en un único bit, igual que veníamos haciendo con enteros con signo.
- La **mantisa** (a veces llamada significando) es la parte $1.f$, donde el 1 antes de la coma siempre está (por eso ni siquiera hace falta guardarlo, es un bit “implícito”) y f son los bits fraccionarios que se guardan explícitamente. La mantisa nos da la *precisión* del número, es decir, cuántas cifras significativas podemos distinguir.
- El **exponente** (e) nos dice cuánto hay que correr la coma binaria, y nos da el *rango*, es decir, qué tan grande o tan chico puede ser el número que representamos.

Esto es exactamente lo mismo que separar, en la notación científica decimal, la información de “qué dígitos tiene el número” (mantisa) de la información de “qué tan grande o chico es” (exponente).

8.2. Por qué existe el estándar IEEE 754

Antes de 1985, cada fabricante de hardware implementaba la aritmética de punto flotante a su manera: elegía cuántos bits usar para el exponente y la mantisa, cómo redondear, cómo manejar el cero o los casos límite, sin ponerse de acuerdo con los demás fabricantes. Esto traía un problema serio: el mismo cálculo, con los mismos números, podía dar resultados distintos según en qué computadora se ejecutara, y un programa que funcionaba bien en una máquina podía comportarse distinto al correrlo en otra.

El estándar **IEEE 754**, publicado en 1985, resuelve este problema fijando de manera precisa cómo se codifican los números en punto flotante (cuántos bits para el signo, el exponente y la mantisa, y cómo se interpretan) y cómo deben comportarse las operaciones aritméticas. Hoy es el estándar que usa prácticamente todo el hardware y todos los lenguajes de programación, así que cuando hablamos de un **float** o un **double** en cualquier lenguaje, nos estamos refiriendo a los formatos de precisión simple (32 bits) y precisión doble (64 bits) que define IEEE 754.

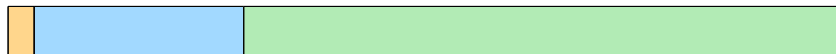
8.3. Normalización

Vimos que cualquier número (salvo el 0) se puede escribir como $\pm 1.f \times 2^e$. Escribir un número de esta forma, con un único 1 antes de la coma, se llama **normalizar** el número.

La normalización cumple dos propósitos. Primero, hace que la representación de cada número sea única: $1,101 \times 2^3$ y $11,01 \times 2^2$ representan el mismo valor, pero solo la primera forma está normalizada, así que no hay ambigüedad sobre cuál usar. Segundo, como en base 2 el único dígito distinto de 0 es el 1, el bit antes de la coma en un número normalizado siempre vale 1. Por eso ni siquiera hace falta guardarlo: es el bit **implícito** que mencionamos antes, y nos ahorra un bit entero de almacenamiento.

8.4. Estructura de un número IEEE 754

Un número en IEEE 754 se guarda en tres campos consecutivos, en este orden: **signo**, **exponente** y **mantisa**. Así se ve el reparto de bits en precisión simple:



■ Signo (1 bit) ■ Exponente ■ Mantisa

Formato	Bits totales	Bits de exponente	Bits de mantisa
Precisión simple (float)	32	8	23
Precisión doble (double)	64	11	52

El **bit de signo** funciona igual que en enteros: 0 para positivo, 1 para negativo. Los **bits de exponente** determinan el rango de valores representables, y los **bits de mantisa** determinan la precisión (cuántas cifras significativas distinguimos).

Hay un detalle importante en cómo se guarda el exponente. El exponente e puede ser negativo, por ejemplo para representar números menores a 1, pero en vez de reservar un bit de signo aparte para el exponente, IEEE 754 le suma un valor fijo llamado **sesgo** (*bias*) antes de guardarlo. En precisión simple el sesgo es 127, en precisión doble es 1023. Es decir que en el campo de exponente no se guarda e directamente, sino

$$e_{\text{guardado}} = e + \text{sesgo}$$

De esta forma el campo de exponente se guarda siempre como un número sin signo, sin necesitar ningún esquema especial para representar exponentes negativos.

9. Cómo convertir un número decimal a IEEE 754

Veamos el procedimiento completo para pasar un número decimal con parte fraccionaria al formato de precisión simple (32 bits, sesgo 127). El procedimiento para precisión doble es el mismo, solo cambia la cantidad de bits de exponente y mantisa, y el sesgo.

Ejemplo. Convertir 13,375 a IEEE 754 de precisión simple.

Paso 1: el signo. El número es positivo, entonces el bit de signo es 0.

Paso 2: convertir la parte entera a binario. Usamos el método de la sección 4.

$$13_{10} = 1101_2$$

Paso 3: convertir la parte fraccionaria a binario. Multiplicamos la parte fraccionaria por 2, nos quedamos con la parte entera del resultado como el próximo bit, y seguimos con la parte fraccionaria que queda, hasta llegar a 0.

$$\begin{aligned} 0,375 \times 2 &= 0,75 &\rightarrow \text{bit } 0 \\ 0,75 \times 2 &= 1,5 &\rightarrow \text{bit } 1 \\ 0,5 \times 2 &= 1,0 &\rightarrow \text{bit } 1 \end{aligned}$$

La parte fraccionaria llegó a 0, así que terminamos. Leyendo los bits en el orden en que salieron obtenemos

$$0,375_{10} = 0,011_2$$

Paso 4: juntar ambas partes.

$$13,375_{10} = 1101,011_2$$

Paso 5: normalizar. Corremos la coma hasta que quede un único 1 antes de ella, y contamos cuántos lugares corrimos, ese número es el exponente.

$$1101,011_2 = 1,101011_2 \times 2^3$$

Corrimos la coma 3 lugares hacia la izquierda, entonces $e = 3$.

Paso 6: calcular el exponente con sesgo.

$$e_{\text{guardado}} = 3 + 127 = 130$$

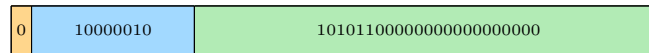
Convertimos 130 a binario con 8 bits, usando de nuevo el método de la sección 4:

$$130_{10} = 10000010_2$$

Paso 7: la mantisa. Nos quedamos con los bits que están después de la coma en la forma normalizada (el 1 antes de la coma no se guarda, es el bit implícito), y completamos con ceros a la derecha hasta llegar a 23 bits.

$$101011 \longrightarrow 10101100000000000000000$$

Paso 8: armar el número completo. Concatenamos **signo**, **exponente con sesgo** y **mantisa**, en ese orden:



Esos 32 bits son la representación de 13,375 en IEEE 754 de precisión simple.

Un comentario importante

Elegimos 0,375 porque su parte fraccionaria en binario termina exactamente, da 0,011 y no sigue más. Esto no siempre pasa: hay números decimales, como 0,1, cuya parte fraccionaria en binario es infinita y periódica, y que por lo tanto no se pueden representar de forma exacta con una cantidad finita de bits de mantisa. En esos casos hay que cortar la mantisa en el último bit disponible, y ese corte es una fuente de error de representación que se estudia más adelante.